

MANUAL DE GIT PARA EL EQUIPO

Proyecto Java — POO con JDBC y Swing

Guía para trabajar en equipo con GitHub

main	develop	feature/...
Código estable final. Nadie toca esta rama directamente.	Rama de integración del equipo. Base para trabajar.	Tu rama personal. Una por cada tarea que hagas.

1. Clonar el repositorio (SOLO UNA VEZ)

Haces esto una sola vez en tu computadora. Te descarga todo el proyecto de GitHub.

```
git clone https://github.com/BryanLS736/ProyectoP00_PanaderiaMarcela.git
# Luego entra a la carpeta del proyecto:
cd ProyectoP00_PanaderiaMarcela.git
git checkout develop
git branch --set-upstream-to=origin/develop develop
```

OJO

Después del clone, ya no vuelves a correr ese comando. Para actualizar tu código usa: `git pull`.

2. Antes de empezar a trabajar (siempre)

Cada vez que vayas a trabajar, primero baja los cambios que hicieron tus compañeros.

1

Ir a la rama develop

```
git checkout develop
```

2

Bajar los últimos cambios

```
git pull
```

3

Crear tu rama para la tarea

Cambia el nombre según tu tarea, De lo siguiente siempre va a tener la siguiente estructura:

el nombre-tarea va a cambiar según la tarea de cada uno.

```
git checkout -b feature/nombre-tarea
```

A continuación, se les muestra ejemplos, donde solo va a variar a lo que le sigue de feature/:

```
git checkout -b feature/ventana-login
```

BIEN

Siempre parte de develop actualizado antes de crear tu rama. Así evitas conflictos.

3. Mientras trabajas

Mientras programas, guarda tu avance en Git con commits. Hazlo seguido, no esperes a terminar todo.

1

Ver qué archivos cambiaste

```
git status
```

2

Agregar tus cambios

```
# Agregar todos los archivos modificados:
```

```
git add .
```

3

Guardar el commit con un mensaje claro

La estructura es:
`git commit -m "Mensaje"`

Se le pone un Mensaje detallando lo que se hizo

Por ejemplo:

`git commit -m "agrego formulario de login con validación"`

Mensajes malos	Mensajes buenos
"cambios"	"agrego ventana de registro"
"arregle cosas"	"corrijo error al conectar a MySQL"
"avance"	"completo CRUD de productos"
"fix"	"valido campos vacíos en formulario"

4

Subir tu rama a GitHub

Se sube la rama con lo siguiente:

`git push`

4. Cuando terminas tu tarea

1

Abrir un Pull Request en GitHub

1. Ve al repositorio en GitHub.com
2. Aparecerá un botón amarillo "Compare & pull request" — haz clic
3. Asegúrate que va de tu rama → develop (NO a main) SI EN CASO TIENES DUDAS, PREGÚNTALE AL LIDER
4. Escribe un título claro y haz clic en "Create pull request"
5. Espera a que el líder lo apruebe y lo una a develop

OJO

NUNCA hagas el Pull Request hacia main. SIEMPRE hacia develop.

5. Para empezar una nueva tarea

Cuando termines una tarea y hagas el Pull Request, empieza la siguiente desde cero con una rama nueva.

```
# Regresa a develop y actualiza  
git checkout develop  
git pull  
  
# Crea una nueva rama para tu siguiente tarea  
git checkout -b feature/nombre-nueva-tarea
```

REGLA

Una rama por tarea. NO REUTILICES RAMAS ANTIGUAS.

6. Resumen del flujo diario

#	Comando	Para qué sirve
1	git checkout develop	Ir a la rama de integración
2	git pull	Bajar cambios de tus compañeros
3	git checkout -b feature/nombre_detu_tarea	Crear tu rama personal
4	git add .	Preparar tus archivos para guardar
5	git commit -m "mensaje_detallado"	Guardar un punto de control
6	git push	Subir tu trabajo a GitHub
7	Pull Request en GitHub	Pedir que tu código se una a develop

7. Reglas de oro del equipo

	NUNCA hagas push directo a main ni a develop
	NUNCA trabajes en la rama de otro sin avisarle
	SIEMPRE haz git pull antes de empezar a trabajar
	SIEMPRE crea una rama nueva por cada tarea
	SIEMPRE escribe mensajes de commit claros y descriptivos
	SIEMPRE espera aprobación antes de fusionar tu PR

¿Dudas? Pregúntale al líder del proyecto antes de hacer algo que no estés seguro.